

PRÁCTICA AUTÓNOMA

Tipos de Datos y Constantes en PSeInt

Sesión 7 | jueves 24 de Septiembre 2026

Módulo: Diseño de Algoritmos (CSTI12010)

Facilitador: Giovanni Antonio Coto Calderón

INSTRUCCIONES

A continuación encontrará 8 ejercicios progresivos de dificultad. Debe resolver cada uno en PSeInt siguiendo estas indicaciones:

1. Leer cuidadosamente el enunciado y requisitos
2. Declarar las variables necesarias con sus tipos de datos
3. Usar constantes donde sea apropiado
4. Comentar el código explicando QUÉ hace cada línea
5. Probar con los datos sugeridos
6. Guardar con nombre: S07_Ejercicio_N.psc (donde N es el número)

EJERCICIO 1: Declaración de Variables (Básico)

Dificultad: ★ Fácil

Enunciado:

Crear un programa que declare variables para almacenar información de un libro: título (Carácter), autor (Carácter), año de publicación (Entero), y precio (Real). Luego, asigne valores y muestre la información formateada.

Requisitos:

Usar los 4 datos: título, autor, año, precio

Cada variable debe tener su tipo correcto

Mostrar la información de forma ordenada

Datos de entrada sugeridos:

Título: "Cien años de soledad"

Autor: "Gabriel García Márquez"

Año: 1967

Precio: 25.50

EJERCICIO 2: Uso de Constantes (Básico-Intermedio)

Dificultad: ★★ Intermedio

Enunciado:

Crear un programa que utilice constantes para calcular el presupuesto de un viaje. Use constantes para el precio de la gasolina por litro, tarifa de hotel por noche, y costo de comidas. Calcule el costo total.

Requisitos:

Definir 3 constantes para precios fijos
Usar variables para cantidad (litros, noches, días)
Calcular costo de gasolina, hotel y comidas
Mostrar desglose detallado

Datos de entrada sugeridos:

Gasolina: 800 pesos/litro, consumo: 50 litros
Hotel: 80 dólares/noche, estancia: 5 noches
Comidas: 30 dólares/día, viaje: 6 días

EJERCICIO 3: Conversión de Tipos (Intermedio)

Dificultad: ★★ Intermedio

Enunciado:

Crear un programa que lea un número como texto, lo convierta a número entero, realice cálculos, y luego convierta el resultado de vuelta a texto para mostrar un mensaje final.

Requisitos:

Usar `ConvertirANumero()` para convertir entrada
Realizar operaciones (suma, multiplicación)
Usar `ConvertirATexto()` para el resultado
Mostrar mensaje que combine tipos de datos

Datos de entrada sugeridos:

Número como texto: "15"
Operaciones: multiplicar por 3 y sumar 10

EJERCICIO 4: Validación de Tipos (Intermedio)

Dificultad: ★★ Intermedio

Enunciado:

Crear un programa que valide si un número entero ingresado está dentro de un rango específico. Use variables lógicas para almacenar resultados de comparaciones.

Requisitos:

Definir constantes para los límites del rango
Usar variables Lógico para almacenar comparaciones
Validar si el número está dentro del rango
Mostrar si es válido o no

Datos de entrada sugeridos:

Rango: 0 a 100
Prueba con: 50 (válido), -10 (inválido), 150 (inválido)

EJERCICIO 5: Cálculo de Cuota Mensual (Intermedio-Avanzado)

Dificultad: ★ ★ ★ Avanzado

Enunciado:

Crear un programa que calcule la cuota mensual de un préstamo. El usuario ingresa el monto del préstamo, la tasa de interés anual y el plazo en meses. Calcule la cuota mensual usando la fórmula apropiada.

Requisitos:

Usar variables Real para montos y tasas

Usar Entero para plazo en meses

Realizar cálculos financieros con precisión

Mostrar resultado formateado

Datos de entrada sugeridos:

Préstamo: 50000 (Real)

Tasa anual: 6.5% (Real)

Plazo: 24 meses (Entero)

EJERCICIO 6: Generador de Contraseña (Intermedio-Avanzado)

Dificultad: ★ ★ ★ Avanzado

Enunciado:

Crear un programa que genere una contraseña combinando el nombre de usuario (Carácter), una constante de prefijo, el año actual (Entero), y un número especial (Entero).

Requisitos:

Leer nombre de usuario (Carácter)

Usar constante para prefijo de seguridad

Combinar diferentes tipos de datos

Convertir números a texto si es necesario

Datos de entrada sugeridos:

Usuario: "carlos_24"

Año: 2026

Número especial: 99

EJERCICIO 7: Sistema de Calificaciones (Avanzado)

Dificultad: ★ ★ ★ ★ Muy Avanzado

Enunciado:

Crear un programa que procese calificaciones de estudiantes. Lee nombre (Carácter), 3 calificaciones (Real), calcula promedio, determina si pasó/reprobó (Lógico) y asigna una letra de calificación (Carácter).

Requisitos:

Usar constante para nota mínima aprobatoria (6.0)

Almacenar el promedio como Real

Usar Lógico para resultado de aprobado/reprobado

Asignar letras: A (9-10), B (7-8.99), C (6-6.99), D (<6)

Mostrar reporte detallado

Datos de entrada sugeridos:

Nombre: "María López"

Notas: 8.5, 7.0, 9.2

EJERCICIO 8: Conversor de Unidades (Muy Avanzado)

Dificultad: ★ ★ ★ ★ Muy Avanzado

Enunciado:

Crear un programa que convierta temperaturas entre Celsius y Fahrenheit. Use constantes para las fórmulas de conversión, y cree un programa que determine si es frío, templado o caliente basándose en rangos de temperatura.

Requisitos:

Definir constantes para factores de conversión

Usar variables Real para temperaturas

Validar rangos de temperatura realistas

Usar Lógico para validaciones

Clasificar temperatura con Carácter (Frío, Templado, Caliente)

Datos de entrada sugeridos:

Celsius: 25 (Templado)

Celsius: 5 (Frío)

Celsius: 35 (Caliente)

RÚBRICA DE EVALUACIÓN

CRITERIO	EXCELENTE (10)	SATISFACTORIO (7-8)	DEFICIENTE (<7)
Tipos de datos	Usa tipos correctos en todas las variables	Usa tipos correctos en mayoría de variables	Confunde tipos de datos
Constantes	Define y usa constantes apropiadamente	Define constantes pero no siempre las usa	No usa constantes cuando debería
Lógica	Código funciona perfectamente con todos los datos	Código funciona con datos normales	Código no funciona correctamente
Comentarios	Comentarios claros en cada línea importante	Algunos comentarios pero incompletos	Muy pocos o sin comentarios

TIPS DE PROGRAMACIÓN

1. Siempre usa nombres descriptivos para variables
2. Las constantes deben estar en MAYÚSCULAS
3. Declara todas las variables al inicio del algoritmo
4. Prueba tu código con varios valores de entrada
5. Usa ConvertirATexto() y ConvertirANumero() cuando sea necesario
6. Mantén tu código limpio y legible con indentación